

1. Basic Function Definition

• [1]:

```
def add(a, b):  
  
    """Add two numbers and  
    return the result."""  
  
    return a + b  
  
result = add(3, 5)  
print(result)  
  
# clcoding.com
```

2. Keyword Parameters

[2]:

```
def greet(name, greeting="Hello"):
    """Greet someone with a provided
    greeting or a default greeting."""
    return f"{greeting}, {name}!"
```

```
message = greet("Clcoding")
print(message)
```

```
message = greet("Python", "Hi")
print(message)
```

```
# clcoding.com
```

```
Hello, Clcoding!
Hi, Python!
```

3. Arbitrary Positional Parameters

- [3]:

```
def sum_all(*args):  
    """Sum all given arguments."""  
    return sum(args)  
  
total = sum_all(1, 2, 3, 4, 5)  
print(total)  
  
# clcoding.com
```

4. Arbitrary Keyword Parameters

•[4]:

```
def print_info(**kwargs):  
  
    """Print key-value pairs  
    from keyword arguments."""  
  
    for key, value in kwargs.items():  
        print(f"{key}: {value}")  
  
print_info(name="Clcoding", age=30, city="Earth")  
  
# clcoding.com
```

```
name: Clcoding  
age: 30  
city: Earth
```

5. Function Annotations

•[5]:

```
def multiply(x: int, y: int) -> int:

    """Multiply two integers
    and return the result."""

    return x * y

result = multiply(3, 4)
print(result)

# clcoding.com
```

6. Docstrings

[6]:

```
def divide(x, y):  
    """  
    Divide x by y and return the result.  
  
    :param x: numerator  
    :param y: denominator  
    :return: division result  
    """  
    return x / y  
  
result = divide(10, 2)  
print(result)  
  
# clcoding.com
```

5.0

7. First-Class Functions

[7]:

```
def square(x):  
    """Return the square of x."""  
    return x * x  
  
def apply_function(func, value):  
    """Apply a function to a value  
    and return the result."""  
    return func(value)  
  
result = apply_function(square, 4)  
print(result)  
  
# clcoding.com
```